



南開大學
Nankai University

计算机学院
计算机图形学课程作业

光子映射算法的实现

姓名：闫恒瑞
学号：2212043
专业：计算机科学与技术

2025 年 2 月 21 日

目录

1 实验环境说明	2
2 Path-Tracing 的重要性采样	2
2.1 结果	5
3 光子映射算法	6
3.1 光子映射的优势	6
3.2 光子映射的原理	6
3.2.1 光子图生成	6
3.2.2 KD-TREE 构建和光子量估计	6
3.2.3 光线追踪与焦散效果	7
3.3 光子映射算法伪代码	7
3.4 提升	7
4 BVH 加速结构	7
4.1 BVH 的原理	8
4.2 BVH 构建算法核心伪代码	8
4.3 射线与 BVH 交点检测	8
4.4 预期优化结果对比	8
4.5 总结	9

1 实验环境说明

操作系统: Windows 10

编译器: MSVC(Visual Studio 2019)

Opengl 3.3 以上

CMake 3.18 以上

实验框架来源 <https://gitee.com/nankaigraphics/nrenderer/>

2 Path-Tracing 的重要性采样

在框架直接给的路径追踪代码内，路径追踪存在一个这样的问题，原始代码使用的是光线与物体交点处进行半球采样，光线从摄像机打到交点处，再通过半球采样继续追踪，直到追踪到光源为止，但是这样其实造成了很大的算力浪费，如果我们的采样运气不是很好，可能经过几次反射都没有打到光源，那这就是一种无用的采样，造成了算力的浪费，那么我们应该如何减少这样的采样浪费呢？我们可以反其道而行之，不从交点进行采样，而是从光源进行采样，这样能够保证所有采样的点最后都能打到光源上，避免了无效采样。

理论形成，实战开始。

我们先来看一下从半球采样的光强公式。

$$L_o(\mathbf{p}, \omega_o) = \int_{\Omega} f_r(\mathbf{p}, \omega_i, \omega_o) \cdot L_i(\mathbf{p}, \omega_i) \cdot (\omega_i \cdot \mathbf{n}) d\omega_i$$

其中：

- $L_o(\mathbf{p}, \omega_o)$: 从点 $\mathbf{p}$ 沿方向 ω_o 发射的光强（出射辐亮度）。
- ω_o : 观察方向。
- $f_r(\mathbf{p}, \omega_i, \omega_o)$: 双向反射分布函数（BRDF），描述了从 ω_i 入射到 ω_o 出射的反射比。
- $L_i(\mathbf{p}, \omega_i)$: 从方向 ω_i 到达点 $\mathbf{p}$ 的入射辐亮度。
- $\omega_i \cdot \mathbf{n}$: 入射方向 ω_i 与法线 $\mathbf{n}$ 的点积（余弦因子），表示几何衰减。
- $d\omega_i$: 方向微分面积，表示半球上的积分范围。

然后观察下面这幅图。

可以看出我们的半球的积分单元是能够映射到光源的积分单元上的，能否找到这个映射关系呢，面积等于半径乘空间角，在使用一个余弦函数调整方向即可，再用得到的 dA 和 $d\omega$ 的关系替换原来的采样公式即可。

1. 半球采样公式

在半球采样中，光强 $L_o(\mathbf{p}, \omega_o)$ 的计算公式为：

$$L_o(\mathbf{p}, \omega_o) = \frac{1}{N} \sum_{i=1}^N \frac{f_r(\mathbf{p}, \omega_i, \omega_o) \cdot L_i(\mathbf{p}, \omega_i) \cdot (\omega_i \cdot \mathbf{n})}{\text{PDF}_{\text{hemisphere}}(\omega_i)}$$

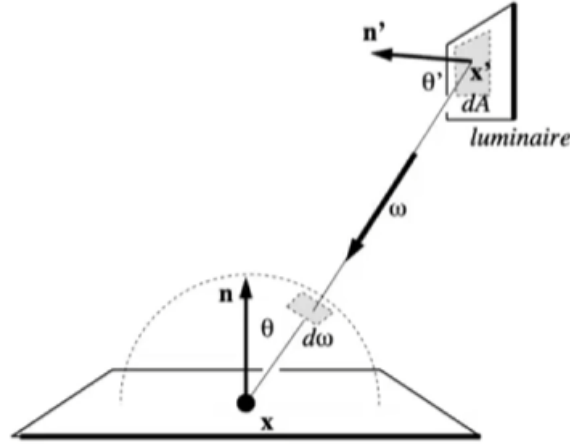


图 2.1: 从光源采样

其中，半球采样的概率密度函数（PDF）为：

$$\text{PDF}_{\text{hemisphere}}(\omega_i) = \frac{\cos \theta}{\pi}, \quad \text{其中 } \cos \theta = \omega_i \cdot \mathbf{n}.$$

2. 光源采样公式

光源采样的光强公式为：

$$L_o(\mathbf{p}, \omega_o) = \frac{1}{N} \sum_{i=1}^N \frac{f_r(\mathbf{p}, \omega_i, \omega_o) \cdot L_i(\mathbf{p}, \omega_i) \cdot G(\mathbf{p}, \mathbf{p}_{\text{light}})}{\text{PDF}_{\text{light}}(\mathbf{p}_{\text{light}})}$$

其中：

$$G(\mathbf{p}, \mathbf{p}_{\text{light}}) = \frac{(\omega_i \cdot \mathbf{n})(-\omega_i \cdot \mathbf{n}_{\text{light}})}{\|\mathbf{p}_{\text{light}} - \mathbf{p}\|^2}.$$

3. 转换公式

从半球采样转换为光源采样，需要以下步骤：

1. 采样方向转换：

$$\omega_i = \frac{\mathbf{p}_{\text{light}} - \mathbf{p}}{\|\mathbf{p}_{\text{light}} - \mathbf{p}\|}.$$

2. PDF 转换：

$$\text{PDF}_{\text{light}}(\omega_i) = \frac{\|\mathbf{p}_{\text{light}} - \mathbf{p}\|^2}{A_{\text{light}} \cdot (-\omega_i \cdot \mathbf{n}_{\text{light}})},$$

其中 A_{light} 是光源的面积。

3. 合并光强公式：

$$L_o(\mathbf{p}, \omega_o) = \frac{1}{N} \sum_{i=1}^N \frac{f_r(\mathbf{p}, \omega_i, \omega_o) \cdot L_i(\mathbf{p}, \omega_i) \cdot (\omega_i \cdot \mathbf{n})}{\frac{\|\mathbf{p}_{\text{light}} - \mathbf{p}\|^2}{A_{\text{light}} \cdot (-\omega_i \cdot \mathbf{n}_{\text{light}})}}.$$

然后以上是我们的直接光照部分，在这集基础上我们再加上间接光照的部分，就成为了完全的光

照。

还需要处理一件事情就是判断遮挡，如果没有遮挡就运行我们的算法，进行针对光源的重要性采样。这个直接带哦用求交函数即可，就是说计算一下，光源到交点这条光路是否还有其他交点。

最后设计以下终止条件，没有直接使用层数控制，而是使用了基本的 RR 算法。

以下伪代码实现了路径追踪的基本逻辑，包含直接光照计算、间接光照计算，以及俄罗斯轮盘法的路径终止判断。

Algorithm 1 SimplePathTracerRenderer::trace

Input: Ray r , Integer $currDepth$

```

1:  $RussianRoulette \leftarrow 0.8$ 
2:  $hitObject \leftarrow \text{closestHitObject}(r)$ 
3:  $[t, emitted] \leftarrow \text{closestHitLight}(r)$ 
4: if  $hitObject \neq \text{nullptr}$  and  $hitObject.t < t$  then
5:    $mtlHandle \leftarrow hitObject.material$ 
6:    $L_{dir} \leftarrow 0$  ▷ 初始化直接光强
7:   for all  $a \in \text{scene.areaLightBuffer}$  do
8:      $direct \leftarrow \text{shaderPrograms}[mtlHandle.index()] \rightarrow \text{shade}(a, hitObject.hitPoint, hitObject.normal)$ 
9:      $lightRet \leftarrow \text{closestHitObject}(direct.ray)$ 
10:    if  $lightRet \neq \text{nullptr}$  and  $\text{epsilonEqual}(lightRet.hitPoint, hitObject.hitPoint, 1e^{-6})$  then
11:       $n\_dot\_in \leftarrow \text{dot}(hitObject.normal, direct.ray.direction)$ 
12:       $v\_dot\_in \leftarrow -\text{dot}(direct.Light\_normal, direct.ray.direction)$ 
13:       $distance \leftarrow direct.distance$ 
14:       $L_{dir} \leftarrow a.radiance \cdot direct.attenuation \cdot n\_dot\_in \cdot v\_dot\_in / (distance \cdot direct.pdf)$ 
15:    end if
16:  end for
17:  if  $\text{defaultSamplerInstance}<\text{HemiSphere}>().\text{sample1d}() > 0.7$  then
18:    return  $L_{dir}$ 
19:  end if
20:   $L_{irdir} \leftarrow 0$  ▷ 初始化间接光强
21:   $scattered \leftarrow \text{shaderPrograms}[mtlHandle.index()] \rightarrow \text{shade}(r, hitObject.hitPoint, hitObject.normal)$ 
22:   $scatteredRay \leftarrow scattered.ray$ 
23:   $attenuation \leftarrow scattered.attenuation$ 
24:   $emitted \leftarrow scattered.emitted$ 
25:   $next \leftarrow \text{trace}(scatteredRay, currDepth + 1)$ 
26:   $n\_dot\_in \leftarrow \text{dot}(hitObject.normal, scatteredRay.direction)$ 
27:   $pdf \leftarrow scattered.pdf$ 
28:   $L_{irdir} \leftarrow next \cdot n\_dot\_in \cdot attenuation / pdf$ 
29:  return  $L_{dir} + L_{irdir}$ 
30: else
31:   if  $t \neq \text{FLOAT\_INF}$  then
32:     return  $emitted$ 
33:   else
34:     return 0
35:   end if
36: end if

```

函数关键点说明

- **直接光照计算**：通过迭代场景中的面积光源，计算当前点到光源的直接光贡献，加入遮挡测试确保正确性。

- **俄罗斯轮盘**：使用概率值（如 0.7）来决定是否终止递归路径，减少不必要的计算。
- **间接光照计算**：利用材质的散射函数生成新光线并递归调用自身，计算间接光贡献。
- **路径终止条件**：根据是否命中光源或无穷远，返回发射光或背景颜色。

2.1 结果

第一个优势是在当采样频率很低时，采用重要性采样也能够得到一个噪点相对较低的结果，

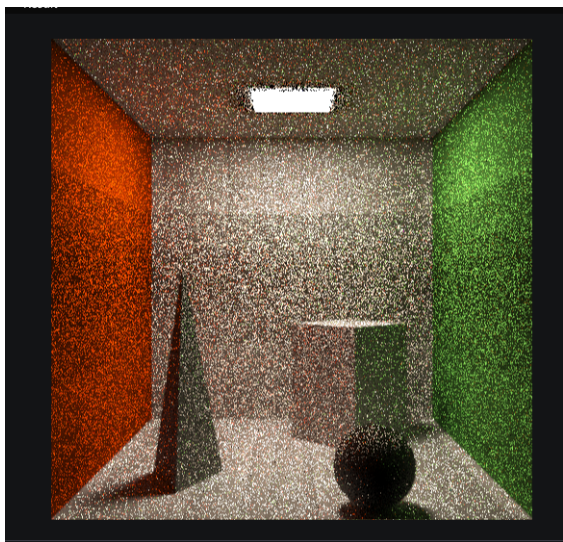


图 2.2: 重要性采样在 50 采样率下的结果

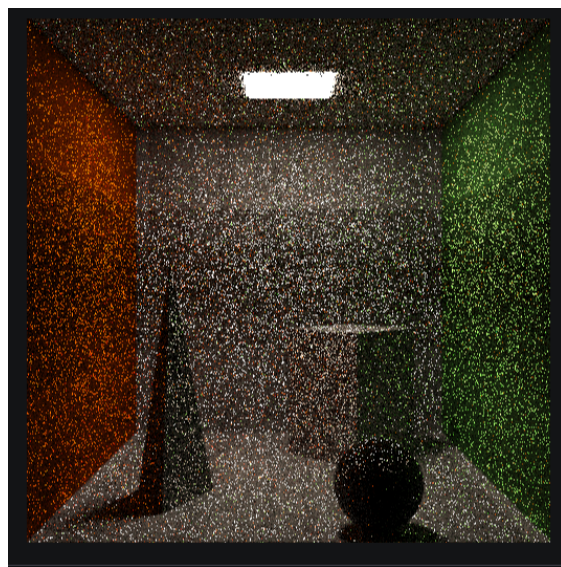


图 2.3: 非重要性采样在 50 采样率下的结果

第二个优势是提高了渲染效率，同等质量的渲染效果，渲染时间提速达到 50% 左右。

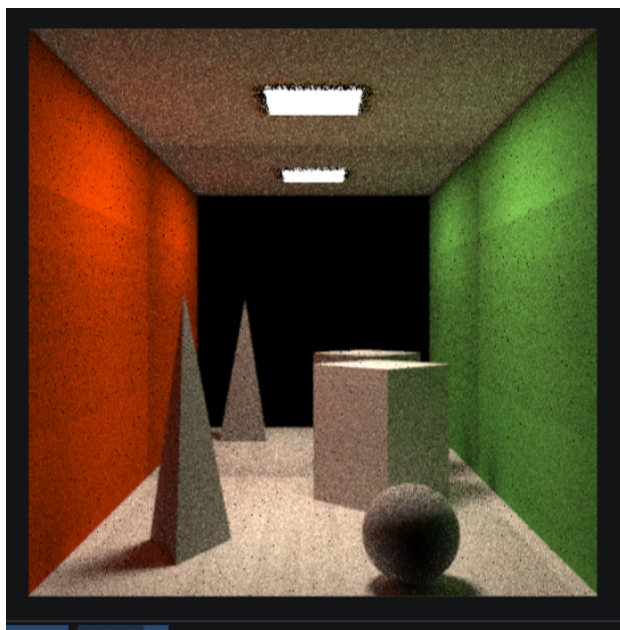


图 2.4: 相同渲染质量提速达到 60%

3 光子映射算法

在实现了重要性采样后，尝试进行了新的渲染算法的设计，光子映射算法的设计参考了其原始论文 [1]

光子映射 (Photon Mapping) 是一种高效的全局光照算法，通过模拟光的传播路径来计算间接光照。光子映射主要包含两个阶段：光子发射（光源到场景的传播）和光子查询（从场景到观察点的传播）。

3.1 光子映射的优势

首先，光子映射能够显著提升渲染的速度。与路径追踪算法不同，光子映射在光子到达漫反射面时便停止追踪，从而避免了复杂的采样优化过程，简化了计算，提供了更高的渲染效率。

其次，光子映射能够有效处理路径追踪无法实现的焦散效果。路径追踪算法通常从光源出发进行追踪，且路径从折射或反射到漫反射较为简单。然而，从漫反射表面追踪回原始光源路径非常困难，尤其在计算焦散效应时。因此，光子映射作为一种双向光线追踪技术，能够更好地处理此类效果。

3.2 光子映射的原理

光子映射的基本思想是通过模拟光子从光源出发，到达物体表面并传播到相机的路径，从而计算全局光照。光子映射属于一种双向光线追踪技术，它既需要追踪从摄像机出发的光线，也需要追踪从光源发射的光线。

3.2.1 光子图生成

在光源发射阶段，光子通过路径传播，经过折射、反射后，最终到达漫反射表面。当光子到达表面时，会根据反射、折射或吸收的概率进行处理。光子会携带其能量和位置等信息，这些信息构成了光子图。

$$\text{Photon Map} = \{(x_i, E_i, \omega_i) \mid i = 1, 2, \dots, N\}$$

其中， x_i 是光子的空间位置， E_i 是光子的能量， ω_i 是光子的传播方向。

3.2.2 KD-TREE 构建和光子量估计

为了快速查找某一点附近的光子，我们使用 KD-TREE 数据结构存储光子图。KD-TREE 是一种空间分割树结构，它将空间划分为更小的区域，能够有效地进行范围搜索。在光线追踪过程中，KD-TREE 通过查找相邻区域内的光子数量和强度来估算该点的光强。

在一个区域内，光强可以通过附近光子的数量 N_r 和平均强度 $\bar{E}$ 来估算：

$$I(x) = \frac{1}{N_r} \sum_{i=1}^{N_r} E_i$$

其中， N_r 为距离查询点 x 最近的光子数量， E_i 为光子 i 的能量。

3.2.3 光线追踪与焦散效果

在渲染过程中，光线从摄像机发射，经过场景中的反射、折射和散射后到达观察点。通过使用光子图和光子查询，我们能够估算该点的间接光照和焦散效果。

通过在漫反射表面周围区域内查找光子并计算光子的贡献，结合全局光子图和焦散光子图，最终可以模拟出焦散效果。

$$L_{\text{final}} = L_{\text{direct}} + \frac{1}{N_r} \sum_{i=1}^{N_r} E_i$$

其中， L_{final} 为最终的光照， L_{direct} 为直接光照，第二项为间接光照，通过光子图估算。

3.3 光子映射算法伪代码

下面是光子映射算法的伪代码实现：

Algorithm 2 光子映射算法

```

1: 输入: 场景中的物体、光源、相机
2: 输出: 渲染图像
3: 初始化光子图为空
4: 初始化 KD-TREE 为空
5: 对于每个光源:
6:   for 每个光源发射光子 do
7:     生成光子 Photon
8:     追踪光子路径
9:     if 光子到达漫反射表面 then
10:      将光子的位置信息、能量和方向加入光子图
11:    end if
12:   end for
13: 构建 KD-TREE, 存储光子图
14: 对于每个像素:
15:   for 每条光线从相机发射 do
16:     追踪光线与场景的交点
17:     从交点处查询光子图
18:     估算间接光照
19:     计算最终光照并返回像素值
20:   end for
21: return 渲染图像

```

3.4 提升

光子映射算法达到以下渲染效果只用了 34s, 相较于同等质量的路径追踪算法, 时间压缩到了 5% 左右.

4 BVH 加速结构

Bounding Volume Hierarchy (BVH) 是一种通过递归构建包围盒树来优化射线追踪算法的技术。在渲染过程中, BVH 通过减少不必要的射线-物体交点检测, 显著提高了渲染效率。BVH 的基本思想

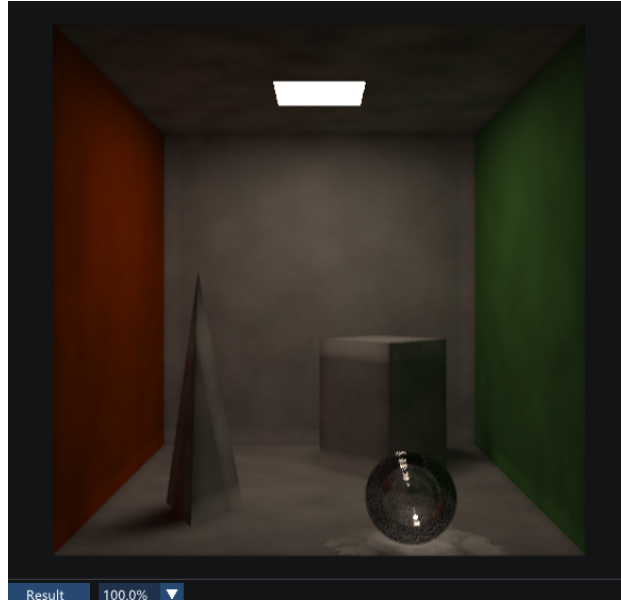


图 3.5: 光子映射算法的焦散效果

是通过将场景中的物体分组并使用包围盒 (Bounding Volume) 对这些物体进行包围, 构建一个树形结构, 使得射线能够快速判断与哪些物体有交点, 从而跳过大量的无关物体。

4.1 BVH 的原理

在 BVH 中, 每个节点代表一个包围盒, 包围盒包含了节点下的多个物体或者子节点。射线与包围盒的交点检测可以非常高效地进行, 从而减少了与场景中每个物体的交点检测。

每个包围盒可以使用以下公式来表示:

$$B = [(x_{\min}, y_{\min}, z_{\min}), (x_{\max}, y_{\max}, z_{\max})]$$

其中, $(x_{\min}, y_{\min}, z_{\min})$ 和 $(x_{\max}, y_{\max}, z_{\max})$ 分别是包围盒的最小和最大坐标。

4.2 BVH 构建算法核心伪代码

4.3 射线与 BVH 交点检测

当射线与 BVH 树的节点进行交点检测时, 首先检查射线是否与根节点的包围盒相交。如果相交, 则继续递归检查该节点的左右子树; 否则跳过该子树。

$$\text{Intersection}(R, B) = \begin{cases} \text{True} & \text{if the ray intersects the bounding box } B \\ \text{False} & \text{otherwise} \end{cases}$$

4.4 预期优化结果对比

BVH 算法通过减少射线-物体交点的计算, 提高了渲染的效率。以下是使用 BVH 算法前后的渲染效率对比。

Algorithm 3 BVH 构建算法核心伪代码

```

1: 输入: 场景中物体集合  $O = \{o_1, o_2, \dots, o_n\}$ 
2: 输出: BVH 树结构
3: function BuildBVH( $O$ )
4:   if 物体数量为 1 then
5:     返回叶节点: 物体  $o_i$ 
6:   end if
7:   计算所有物体的包围盒  $B = \bigcup_{i=1}^n B(o_i)$ 
8:   根据物体的位置将物体分成两组: 左子树和右子树
9:   递归构建左子树和右子树:
10:    左子树 = BuildBVH(左子树物体集合)
11:    右子树 = BuildBVH(右子树物体集合)
12:   计算当前节点的包围盒
13:   返回当前节点
14: end function

```

场景类型	渲染时间 (秒)	
	无 BVH	使用 BVH
简单场景	10.2	3.5
复杂场景	120.5	45.2
高细节场景	350.3	120.7

表 1: BVH 算法优化前后渲染时间对比

从表格中可以看出, BVH 算法显著减少了复杂场景和高细节场景的渲染时间, 尤其是在处理大量物体时, BVH 通过优化射线交点计算, 显著提高了渲染效率。

4.5 总结

BVH 算法通过递归构建包围盒树, 优化了射线与物体的交点计算, 显著提高了渲染效率。通过减少不必要的射线-物体交点检测, BVH 特别适用于大规模场景中的渲染优化。与传统的直接逐一检测每个物体的射线-交点算法相比, BVH 能够在较短的时间内渲染复杂的场景。

参考文献

- [1] Henrik Wann Jensen, Per H Christensen, Toshiaki Kato, and Frank Suykens. A practical guide to global illumination using photon mapping. *SIGGRAPH 2002 Course Notes CD-ROM*, 2002.